

Customer Churn Analysis

Python Notebook — Code, Outputs & Visualizations

Converted from the uploaded Jupyter Notebook. Matplotlib/Seaborn figures are included from the notebook's embedded outputs where available.

Customer Churn Analysis

Code Cell 2

```
#import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import sqlite3
```

sqlite is database system that helps to work with sql database in python

Code Cell 4

```
#import data
conn = sqlite3.connect('customer_churn.db') #connects python with sqlite database
sql_query = """
SELECT name
FROM sqlite_master
WHERE type='table';
"""
tables = pd.read_sql(sql_query,conn)

#create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT*FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"created dataframe: df_{table_name}")
conn.close()
```

Output

```
created dataframe: df_db_customer
created dataframe: df_db_subscription
created dataframe: df_db_support
```

Code Cell 5

```
#print table names and columns names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # get column information
    columns_query = f"PRAGMA table_info({table_name});"
    #PRAGMA table_info() is an SQLite command used to get information about a table's columns.
    columns = pd.read_sql(columns_query,conn)
    print("Columns:")
    print(columns['name'].tolist())
#close connection
conn.close()
```

Output

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'zipcode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_type']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

2. Data Cleaning

first look at customer table

Code Cell 8

```
df_db_customer.head()
```

Output

	customerid	name	country	state	gender	dob	\
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	

	interests	pincode
0	travel	None
1	NaN	None
2	movie	None
3	NaN	None
4	drama	None

```
Code Cell 9

df_db_customer.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null     str
1   name        21 non-null     str
2   country     18 non-null     str
3   state       21 non-null     str
4   gender      21 non-null     str
5   dob         21 non-null     str
6   interests   4 non-null      str
7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

```
Code Cell 10

df_db_customer.isnull().sum()
```

Output

```
customerid    0
name          0
country       3
state         0
gender        0
dob           0
interests     17
pincode       21
dtype: int64
```

```
Code Cell 11

# first rename name column to customer name
df_db_customer.rename(columns = {"name":"customer_name"},inplace=True)
```

```
Code Cell 12

df_db_customer['gender'].unique()
```

Output

```
<ArrowStringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str
```

```
Code Cell 13

# above replace Women into Female and Men into Male
df_db_customer.replace({"Men":"Male","Women":"Female"},inplace=True)
```

Output

	customerid	customer_name	country	state	gender \
0	0002-ORFBO	keshav	India	Maharashtra	Male
1	0003-MKNFE	raghav	India	Karnataka	Male
2	0004-TLHLJ	lalita	India	Delhi	Female
3	0011-IGKFF	mohan	India	Nagaland	Male
4	0013-EXCHZ	mira	India	Delhi	Female
5	0013-MHZWF	durga	NaN	Delhi	Female
6	0013-SMEOE	mina	India	Meghalaya	Female
7	0014-BMAQU	madan	India	Rajasthan	Male
8	0015-UOCOJ	maya	NaN	Kathmandu	Female
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male
10	0017-DINOC	shiva	India	Maharashtra	Male
11	0017-IUDMW	rangadevi	India	Karnataka	Female
12	0018-NYROU	chitra	NaN	Telangana	Female
13	0019-EFAEP	raju	India	Meghalaya	Female
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male
15	0020-INWCK	parvati	India	Delhi	Female
16	0020-JDNXP	rikim	India	Meghalaya	Female
17	0021-IXXGC	vishakha	India	Rajasthan	Female
18	0022-TCJCI	raghvendra	India	Telangana	Male
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male
20	0023-UYUPN	sudevi	India	Maharashtra	Female

	dob	interests	pincode
0	1982-04-12 00:00:00	travel	None
1	1995-11-23 00:00:00	NaN	None
2	1978-02-15 00:00:00	movie	None
3	2001-08-30 00:00:00	NaN	None
4	1990-05-05 00:00:00	drama	None
5	1988-12-10 00:00:00	NaN	None

6	1976-09-21 00:00:00	NaN	None
7	1999-03-14 00:00:00	NaN	None
8	1985-07-07 00:00:00	NaN	None
9	1993-10-29 00:00:00	NaN	None
10	1997-01-22 00:00:00	NaN	None
11	1981-06-18 00:00:00	NaN	None
12	2004-12-01 00:00:00	NaN	None
13	1992-04-25 00:00:00	NaN	None
14	1979-11-11 00:00:00	NaN	None
15	1986-02-28 00:00:00	job	None
16	1994-08-19 00:00:00	NaN	None
17	2000-09-02 00:00:00	NaN	None
18	1983-12-30 00:00:00	NaN	None
19	1991-05-14 00:00:00	NaN	None
20	1977-10-06 00:00:00	NaN	None

Code Cell 14

```
df_db_customer['gender'].unique()
```

Output

```
<ArrowStringArray>
['Male', 'Female']
Length: 2, dtype: str
```

Code Cell 15

```
# here pincode values are all null and interest of customer are not much important for churn analysis.
# so remove pincode and interest columns
df_db_customer.drop(columns=['interests','pincode'],inplace=True)
```

Code Cell 16

```
df_db_customer.head()
```

Output

	customerid	customer_name	country	state	gender	dob
0	0002-ORF80	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00

Code Cell 17

```
df_db_customer.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    str
dtypes: str(6)
memory usage: 2.2 KB
```

Code Cell 18

```
# above you can see that dob column data type should be date but it is in str
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

Code Cell 19

```
df_db_customer.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.8 KB
```

Code Cell 20

```
df_db_customer['state'].unique() # check unique values of state
```

Output

```
<ArrowStringArray>
[ 'Maharashtra',      'Karnataka',      'Delhi',      'Nagaland',
```

```
    'Meghalaya',      'Rajasthan',      'Kathmandu',      'Telangana',
    'Uttar Pradesh']
Length: 9, dtype: str
```

Code Cell 21

```
df_db_customer.isnull().sum() # checking null values
```

Output

```
customerid      0
customer_name    0
country          3
state            0
gender           0
dob              0
dtype: int64
```

Code Cell 22

```
df_db_customer['country'].unique()
```

Output

```
<ArrowStringArray>
['India', nan, 'Nepal']
Length: 3, dtype: str
```

Code Cell 23

```
df_db_customer[df_db_customer['country'].isna()]
```

Output

```
   customerid customer_name country  state gender      dob
5    0013-MHZWF      durga     NaN    Delhi  Female 1988-12-10
8    0015-UOCOJ      maya     NaN  Kathmandu  Female 1985-07-07
12   0018-NYROU    chitra     NaN  Telangana  Female 2004-12-01
```

Code Cell 24

```
# filling correct country name at null places
#country and state - unique value pair
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))
```

Code Cell 25

```
df_db_customer.isnull().sum()
```

Output

```
customerid      0
customer_name    0
country          0
state            0
gender           0
dob              0
dtype: int64
```

now look at the table subscription

Code Cell 27

```
df_db_subscription.head()
```

Output

```
   customerid subscription_start_date subscription_type renewal_date \
0    0002-ORFBO      2021-03-15      Refferal      2025-03-15
1    0003-MKNFE      2020-08-01        Paid      2024-08-01
2    0004-TLHLJ      2022-11-20      Organic      2025-11-20
3    0011-IGKFF      2019-05-10        Paid      2025-05-10
4    0013-EXCHZ      2023-01-05      Refferal      2024-01-05

   plan_type contract_type cancellation_date  cancellation_reason \
0   Standard      Annual           NaN           NaN
1   Premium      Annual      2024-09-10  Switched to competitor
2   Basic       Monthly           NaN           NaN
3   Premium      Annual           NaN           NaN
4   Standard      Monthly      2024-02-28      Too expensive

   monthly_charges  cltv  churn_score
0           13.99    627           12
1           12.99   1150           91
2            6.99    210           34
3           22.99   1725            8
4           13.99    195           88
```

Code Cell 28

```
df_db_subscription.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
```

```

#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB

```

Code Cell 29

```

# above there are few null values in cancellation date which means customer has not cancelled the subscription that is good
# convert date column data type in date
date_colm = ['subscription_start_date', 'renewal_date', 'cancellation_date']
df_db_subscription[date_colm] = df_db_subscription[date_colm].apply(pd.to_datetime)

```

Code Cell 30

```
df_db_subscription.info()
```

Output

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    datetime64[us]
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    datetime64[us]
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     datetime64[us]
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 2.7 KB

```

now look at customer support table

Code Cell 32

```
df_db_support.head()
```

Output

```

   customerid  complaint_date escalations  csat_score col_1 \
0  0003-MKNFE  2024-08-28 00:00:00         N         60  None
1  0003-MKNFE  2024-08-28 00:00:00         Y         10  None
2  0013-EXCHZ  2024-01-20 00:00:00         Y         20  None
3  0013-MHZWF  2025-03-18 00:00:00         N         90  None
4  0013-SMEOE  2024-11-01 00:00:00         N         30  None

   comment
0  service issue
1  demanded refund
2  NaN
3  guidance to renew
4  NaN

```

Code Cell 33

```
df_db_support.info()
```

Output

```

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            9 non-null     str
1   complaint_date        9 non-null     str
2   escalations           9 non-null     str
3   csat_score            9 non-null     int64
4   col_1                 0 non-null     object
5   comment               4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 895.0+ bytes

```

Code Cell 34

```
df_db_support['escalations'].unique()
```

Output

```
<ArrowStringArray>
```

```
['N', 'Y']
Length: 2, dtype: str
```

Code Cell 35

```
# droop col_1 because it is garbage
# drop comments as it not needed much here
df_db_support.drop(columns=['col_1','comment'], inplace=True)
```

Code Cell 36

```
df_db_support.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: int64(1), str(3)
memory usage: 690.0 bytes
```

Code Cell 37

```
#convert date time column data type
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
```

Code Cell 38

```
df_db_support.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     datetime64[us]
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 519.0 bytes
```

3. Feature Engineering and Data analysis

Code Cell 40

```
df_db_subscription.head()
```

Output

```
customerid subscription_start_date subscription_type renewal_date \
0 0002-ORFB0      2021-03-15      Refferal      2025-03-15
1 0003-MKNFE      2020-08-01      Paid      2024-08-01
2 0004-TLHLJ      2022-11-20      Organic      2025-11-20
3 0011-IGKFF      2019-05-10      Paid      2025-05-10
4 0013-EXCHZ      2023-01-05      Refferal      2024-01-05

plan_type contract_type cancellation_date      cancellation_reason \
0 Standard      Annual      NaT      NaN
1 Premium      Annual      2024-09-10      Switched to competitor
2 Basic      Monthly      NaT      NaN
3 Premium      Annual      NaT      NaN
4 Standard      Monthly      2024-02-28      Too expensive

monthly_charges cltv churn_score
0      13.99      627      12
1      12.99     1150      91
2       6.99      210      34
3      22.99     1725       8
4      13.99      195      88
```

Code Cell 41

```
# create a new column - churn flag
#churn flag column which 1 for cancellation and 0 for not cancellation
df_db_subscription['churn flag'] = np.where(df_db_subscription['cancellation_date'].notna(),1,0)
```

Code Cell 42

```
df_db_subscription.head(3)
```

Output

```
customerid subscription_start_date subscription_type renewal_date \
0 0002-ORFB0      2021-03-15      Refferal      2025-03-15
1 0003-MKNFE      2020-08-01      Paid      2024-08-01
2 0004-TLHLJ      2022-11-20      Organic      2025-11-20

plan_type contract_type cancellation_date      cancellation_reason \
```

0	Standard	Annual	NaT	NaN
1	Premium	Annual	2024-09-10	Switched to competitor
2	Basic	Monthly	NaT	NaN

	monthly_charges	cltv	churn_score	churn flag
0	13.99	627	12	0
1	12.99	1150	91	1
2	6.99	210	34	0

Code Cell 43

```
df_db_customer.columns
```

Output

```
Index(['customerid', 'customer_name', 'country', 'state', 'gender', 'dob'], dtype='str')
```

Code Cell 44

```
df_db_support.columns
```

Output

```
Index(['customerid', 'complaint_date', 'escalations', 'csat_score'], dtype='str')
```

Code Cell 45

```
# now we will merge three tables using customerid
df = (df_db_subscription
      .merge(df_db_customer,on = 'customerid',how = 'left')
      .merge(df_db_support,on = 'customerid', how = 'left'))
```

Code Cell 46

```
df.head()
```

Output

	customerid	subscription_start_date	subscription_type	renewal_date	\
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	
2	0003-MKNFE	2020-08-01	Paid	2024-08-01	
3	0004-TLHLJ	2022-11-20	Organic	2025-11-20	
4	0011-IGKFF	2019-05-10	Paid	2025-05-10	

	plan_type	contract_type	cancellation_date	cancellation_reason	\
0	Standard	Annual	NaT	NaN	
1	Premium	Annual	2024-09-10	Switched to competitor	
2	Premium	Annual	2024-09-10	Switched to competitor	
3	Basic	Monthly	NaT	NaN	
4	Premium	Annual	NaT	NaN	

	monthly_charges	cltv	churn_score	churn flag	customer_name	country	\
0	13.99	627	12	0	keshav	India	
1	12.99	1150	91	1	raghav	India	
2	12.99	1150	91	1	raghav	India	
3	6.99	210	34	0	lalita	India	
4	22.99	1725	8	0	mohan	India	

	state	gender	dob	complaint_date	escalations	csat_score
0	Maharashtra	Male	1982-04-12	NaT	NaN	NaN
1	Karnataka	Male	1995-11-23	2024-08-28	N	60.0
2	Karnataka	Male	1995-11-23	2024-08-28	Y	10.0
3	Delhi	Female	1978-02-15	NaT	NaN	NaN
4	Nagaland	Male	2001-08-30	NaT	NaN	NaN

Code Cell 47

```
# very very important
df.shape
```

Output

```
(23, 20)
```

Code Cell 48

```
df_db_subscription.shape
```

Output

```
(21, 12)
```

Code Cell 49

```
df_db_support.shape
```

Output

```
(9, 4)
```

Code Cell 50

```
df_db_customer.shape
```

Output

```
(21, 6)
```

Code Cell 51

```
# in subscription table there are only 21 rows while after left merging 2 more rows added means duplication in data
df_db_subscription['customerid'].nunique() # it is not changing after merging
```

Output

21

Code Cell 52

```
df_db_customer['customerid'].nunique() # after merging it is also right
```

Output

21

Code Cell 53

```
df_db_support['customerid'].nunique() # here in this 2 rows increases
```

Output

7

Code Cell 54

```
df_db_support
```

Output

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

Code Cell 55

```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].transform('count')
```

Code Cell 56

```
df_db_support['complaint_count'] # here it gives the count according to the duplicate customerid
```

Output

0	2
1	2
2	1
3	1
4	1
5	1
6	1
7	2
8	2

Name: complaint_count, dtype: int64

Code Cell 57

```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid', keep = 'last')
# drop duplicate colmid and keep only last value from duplicate value
```

Code Cell 58

```
df_db_support['complaint_count'] # check the duplicate values. is it removed ?
```

Output

2	1
5	1
1	2
8	2
6	1
4	1
3	1

Name: complaint_count, dtype: int64

Code Cell 59

```
df_db_support
```

Output

	customerid	complaint_date	escalations	csat_score	complaint_count
2	0013-EXCHZ	2024-01-20	Y	20	1
5	0017-IUDMW	2024-04-10	Y	25	1
1	0003-MKNFE	2024-08-28	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2
6	0019-EFAEP	2024-09-27	Y	30	1
4	0013-SMEOE	2024-11-01	N	30	1
3	0013-MHZWF	2025-03-18	N	90	1

Code Cell 60

```
# now merge the tables again
df = (df_db_subscription
      .merge(df_db_customer,on = 'customerid', how = 'left')
      .merge(df_db_support, on = 'customerid', how = 'left'))
```

Code Cell 61

```
df.shape
```

Output

```
(21, 21)
```

Code Cell 62

```
df_db_subscription.shape
```

Output

```
(21, 12)
```

Code Cell 63

```
# above as can see now rows are not duplicated (no extra row)
```

Code Cell 64

```
# lets convert df into csv file to view the data in excel
df.to_csv("exported_churn_data.csv",index=False)
```

Data Analysis

churn rate -> how many customer going to leave

Code Cell 67

```
df.columns
```

Output

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

churn rate = (no. of churn customers)/(total customers)

or you can say that mean

Code Cell 69

```
#1. churn rate
churn_rate = df['churn flag'].mean()*100
print("churn rate:", round(churn_rate,2), "%")
```

Output

```
churn rate: 28.57 %
```

Code Cell 70

```
#2. retention rate -> how many are not churn customers
retention_rate = 100 - churn_rate
print("retention rate:", round(retention_rate,2), "%")
```

Output

```
retention rate: 71.43 %
```

Code Cell 71

```
df.head(2)
```

Output

```
customerid subscription_start_date subscription_type renewal_date \
0 0002-ORFBO 2021-03-15 Refferal 2025-03-15
1 0003-MKNFE 2020-08-01 Paid 2024-08-01

plan_type contract_type cancellation_date cancellation_reason \
0 Standard Annual NaT NaN
1 Premium Annual 2024-09-10 Switched to competitor

monthly_charges cltv ... churn flag customer_name country state \
0 13.99 627 ... 0 keshav India Maharashtra
1 12.99 1150 ... 1 raghav India Karnataka

gender dob complaint_date escalations csat_score complaint_count
0 Male 1982-04-12 NaT NaN NaN NaN
1 Male 1995-11-23 2024-08-28 Y 10.0 2.0
```

```
[2 rows x 21 columns]
```

Code Cell 72

```
df['plan_type'].unique()
```

Output

```
<ArrowStringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
```

Code Cell 73

```
# now try to find from which plan most of the customer are churning and what are the loss
churn_by_plan = (df.groupby('plan_type')['churn flag'].mean().mul(100).round(2).reset_index(name='churn_rate_pct'))
print(churn_by_plan)
```

Output

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

Code Cell 74

```
df[df['churn flag'] == 1].groupby('plan_type')['monthly_charges'].sum().reset_index(name='total rev loss')
```

Output

	plan_type	total rev loss
0	Basic	38.97
1	Premium	12.99
2	Standard	21.98

Code Cell 75

```
df[df['churn flag'] == 1].groupby('plan_type')['plan_type'].value_counts().reset_index(name='no. of churn user')
```

Output

	plan_type	no. of churn user
0	Basic	3
1	Premium	1
2	Standard	2

Code Cell 76

```
# summary of the churn by each plan type
plan_summary = (
    df.groupby('plan_type')
        .agg(
            total_customers=('churn flag', 'count'),
            churned_customers=('churn flag', 'sum'),
            churn_rate_pct=('churn flag', lambda x: round(x.mean() * 100, 2)),
            monthly_revenue=('monthly_charges', 'sum'),
            revenue_loss=('monthly_charges',
                          lambda x: x[df.loc[x.index, 'churn flag'] == 1].sum())
        )
        .reset_index()
)

print(plan_summary)
```

Output

	plan_type	total_customers	churned_customers	churn_rate_pct	\
0	Basic	5	3	60.00	
1	Premium	7	1	14.29	
2	Standard	9	2	22.22	

	monthly_revenue	revenue_loss
0	52.95	38.97
1	218.93	12.99
2	123.91	21.98

In this dataset, Basic has the highest observed churn rate (60%) and the highest monthly revenue loss among the three plans.

Code Cell 78

```
# summary of the churn by each state
state_summary = (
    df.groupby('state')
        .agg(
            total_customers=('churn flag', 'count'),
            churned_customers=('churn flag', 'sum'),
            churn_rate_pct=('churn flag', lambda x: round(x.mean() * 100, 2)),
            monthly_revenue=('monthly_charges', 'sum'),
            revenue_loss=('monthly_charges',
                          lambda x: x[df.loc[x.index, 'churn flag'] == 1].sum())
        )
        .reset_index()
)

print(state_summary)
```

Output

	state	total_customers	churned_customers	churn_rate_pct	\
0	Delhi	4	1	25.00	
1	Karnataka	2	2	100.00	
2	Kathmandu	2	0	0.00	
3	Maharashtra	3	0	0.00	
4	Meghalaya	3	2	66.67	
5	Nagaland	1	0	0.00	
6	Rajasthan	2	0	0.00	
7	Telangana	2	1	50.00	
8	Uttar Pradesh	2	0	0.00	

	monthly_revenue	revenue_loss
0	52.96	13.99
1	20.98	20.98
2	20.98	0.00
3	50.97	0.00
4	42.97	21.98
5	22.99	0.00
6	36.98	0.00
7	30.98	16.99
8	115.98	0.00

State-level analysis shows variation in observed churn rates. Karnataka has a 100% churn rate in the dataset, but this is based on only two customers. Meghalaya has a 66.67% churn rate and the highest observed revenue loss among the states shown (as it is a small dataset)

Code Cell 80

```
# summary of the churn by each subscription_type
subscription_summary = (
    df.groupby('subscription_type')
      .agg(
        total_customers=('churn flag', 'count'),
        churned_customers=('churn flag', 'sum'),
        churn_rate_pct=('churn flag', lambda x: round(x.mean() * 100, 2)),
        monthly_revenue=('monthly_charges', 'sum'),
        revenue_loss=('monthly_charges',
                      lambda x: x[df.loc[x.index, 'churn flag'] == 1].sum())
      )
    .reset_index()
)

print(subscription_summary)
```

Output

	subscription_type	total_customers	churned_customers	churn_rate_pct	\
0	Organic	9	0	0.00	
1	Paid	6	1	16.67	
2	Refferal	6	5	83.33	

	monthly_revenue	revenue_loss
0	145.91	0.00
1	174.94	12.99
2	74.94	60.95

Based on the available dataset, the Referral subscription group has the highest observed churn rate at 83.33% (5 out of 6 customers) and the highest revenue loss of 60.95. The Organic group has 0% observed churn, while the Paid group has a 16.67% observed churn rate

Code Cell 82

```
#ARPU -> avg revenue per user
arpu = df['monthly_charges'].mean()
print("ARPU =", round(arpu,2))
```

Output

ARPU = 18.85

Code Cell 83

```
# avg customer Tenure
# count of days user has used our service till cancellation date or current date
today = pd.Timestamp.today()
df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)
avg_tenure_days = df['tenure_days'].mean()
print("average tenure days:", round(avg_tenure_days,0))
```

Output

average tenure days: 1550.0

Code Cell 84

```
# revenue at risk -> revenue lost from churned users
rev_at_risk = df.loc[df['churn flag'] == 1, 'monthly_charges'].sum()
print("revenue at risk (Rs K):", rev_at_risk)
```

Output

```
revenue at risk (Rs K): 73.94
```

Code Cell 85

```
# Escalation rate -> customer who complained but their query not solved
df['escalations'].unique()
```

Output

```
<ArrowStringArray>
[nan, 'Y', 'N']
Length: 3, dtype: str
```

Code Cell 86

```
escalation_rate = (df['escalations'] == 'Y').mean()*100
print("escalation rate:", round(escalation_rate,2), "%")
```

Output

```
escalation rate: 19.05 %
```

Code Cell 87

```
# avg compalint per user
avg_comp = df['complaint_count'].sum()/df['customerid'].nunique()
print("avg complaints per user =", round(avg_comp,2))
```

Output

```
avg complaints per user = 0.43
```

Code Cell 88

```
# correlation Escalation vs churn
df[['escalations', 'churn flag']].dropna()
```

Output

	escalations	churn flag
1	Y	1
4	Y	1
5	N	0
6	N	1
11	Y	1
13	Y	1
18	N	1

Code Cell 89

```
# to find corr we have convert escalation in numeric value (Y -> 1, N -> 0)
df['escalations'] = np.where(df['escalations'] == 'Y',1,0)
corr_df = df[['escalations', 'churn flag']].dropna()
correlation = corr_df['escalations'].corr(df['churn flag'])
```

Code Cell 90

```
print("correlation between escalations vs churn:", round(correlation,2))
```

Output

```
correlation between escalations vs churn: 0.77
```

Code Cell 91

```
#create a column churn risk using churn score
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]
choices = ['low', 'mid', 'high']
df['churn_risk'] = np.select(conditions, choices, default = 'unknown')
df[['churn_score', 'churn_risk']].head()
```

Output

	churn_score	churn_risk
0	12	low
1	91	high
2	34	low
3	8	low
4	88	high

Data Visualization

Code Cell 93

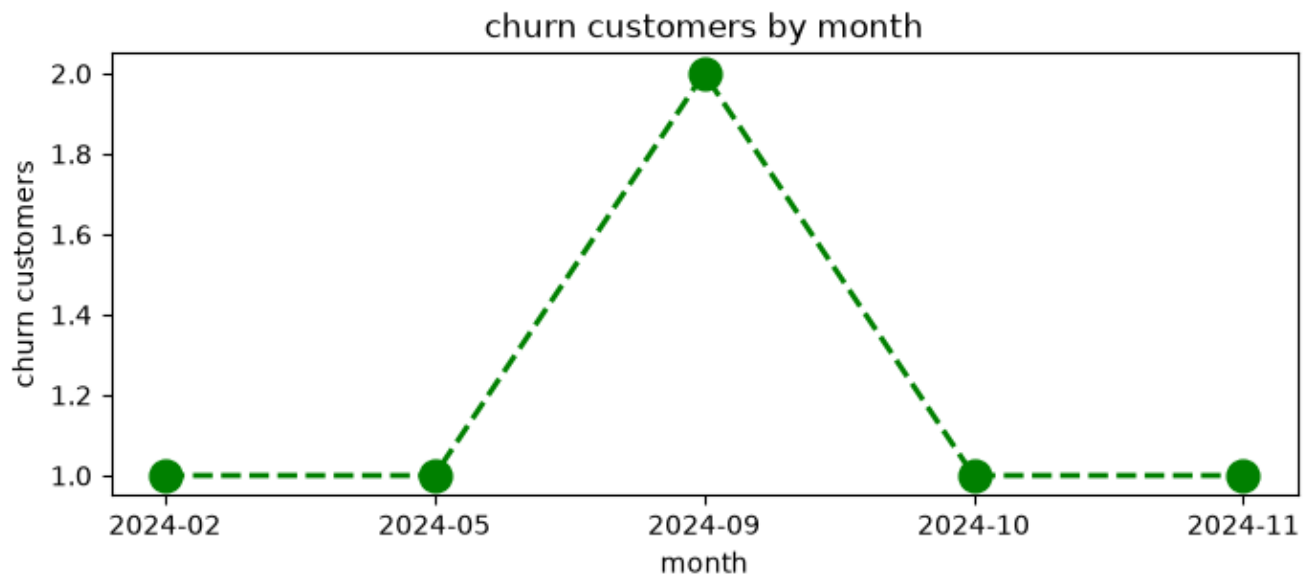
```
df_vis = df.copy() # copy original data so that original data remain as it is
```

Code Cell 94

```
df_vis['cancellation_month'] = df_vis['cancellation_date'].dt.to_period('M')
churn_trend = df_vis[df_vis['churn flag'] == 1].groupby('cancellation_month').size()
plt.figure(figsize=(8,3))
plt.xlabel('month')
```

```
plt.ylabel('churn customers')
plt.title("churn customers by month")
plt.plot(churn_trend.index.astype(str),churn_trend.values, color='green',marker='o',linestyle='dashed',linewidth=2,markersize=12)
plt.show()
```

Visualization

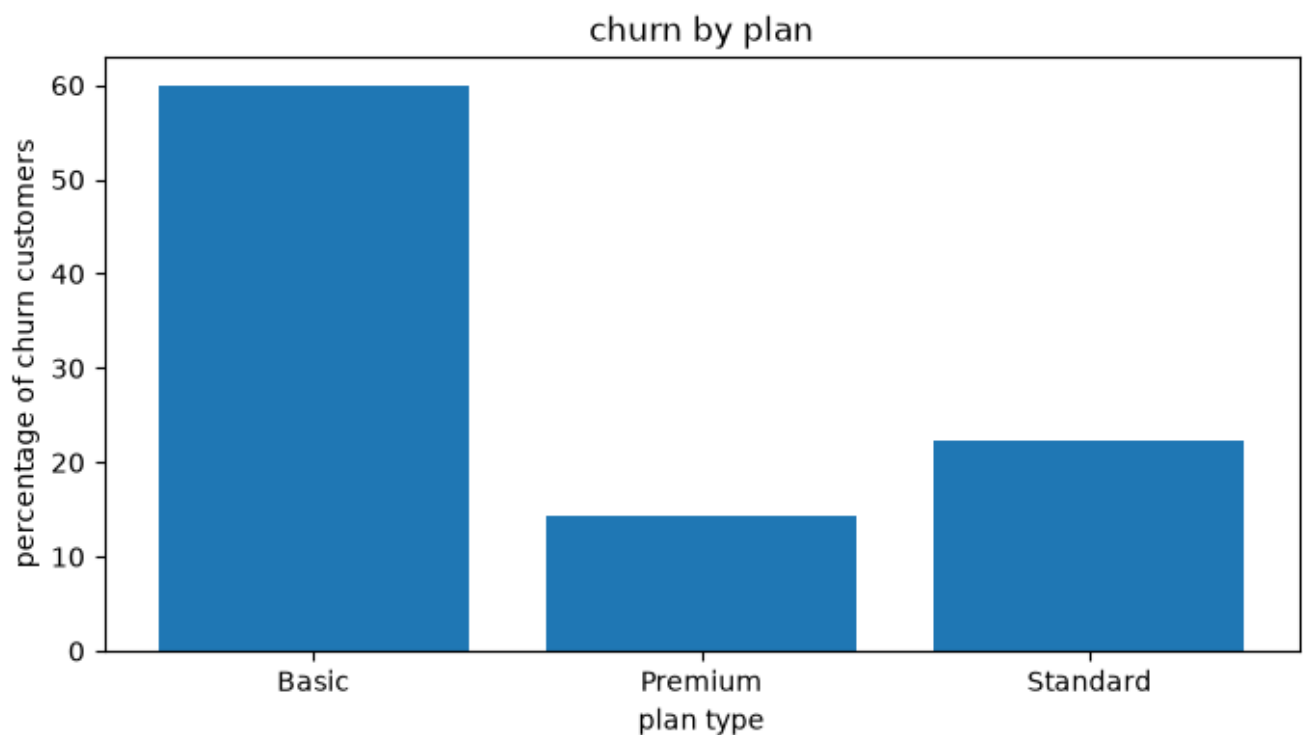


2. Churn by plan type

Code Cell 96

```
churn_plan = df_vis.groupby('plan_type')['churn flag'].mean()*100
plt.figure(figsize=(8,4))
plt.ylabel("percentage of churn customers")
plt.xlabel("plan type")
plt.title("churn by plan")
plt.bar(churn_plan.index,churn_plan.values)
plt.show()
```

Visualization

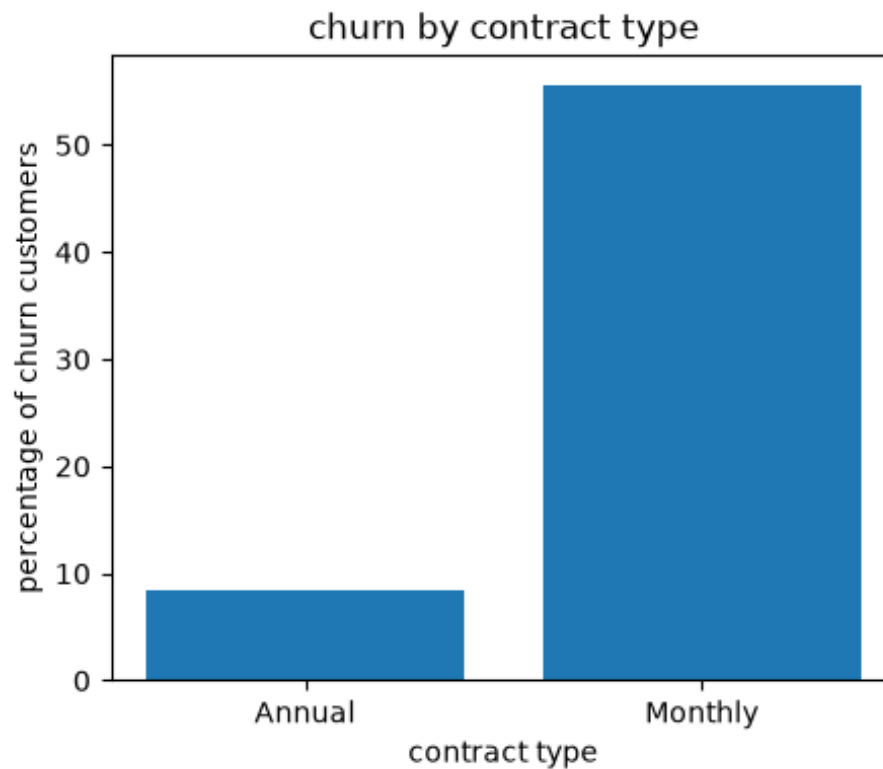


Code Cell 97

```
contract_plan = df_vis.groupby('contract_type')['churn flag'].mean()*100
plt.figure(figsize=(5,4))
plt.ylabel("percentage of churn customers")
plt.xlabel("contract type")
plt.title("churn by contract type")
plt.bar(contract_plan.index,contract_plan.values)
```

```
plt.show()
```

Visualization



Code Cell 98

```
# correlation visualization using seaborn
# encoding -> convert str to numeric so that we can find corr between feaatures
df_vis.columns
```

Output

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

Code Cell 99

```
df_vis[['plan_type', 'contract_type', 'churn_score', 'churn flag', 'churn_risk', 'escalations']].head()
```

Output

	plan_type	contract_type	churn_score	churn flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

Code Cell 100

```
df_encoded = df_vis[['plan_type', 'contract_type', 'churn_score', 'churn flag', 'churn_risk', 'escalations']]
# str to numerical by priority based
order_map = {
    'plan_type': ['Basic', 'Standard', 'Premium'],
    'contract_type': ['Monthly', 'Annual'],
    'churn_risk': ['low', 'mid', 'high']
}
# key idea: String -> Category -> Ordered Category -> Numeric code
for col, order in order_map.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True).codes
```

Code Cell 101

```
df_encoded.head()
```

Output

	plan_type	contract_type	churn_score	churn flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

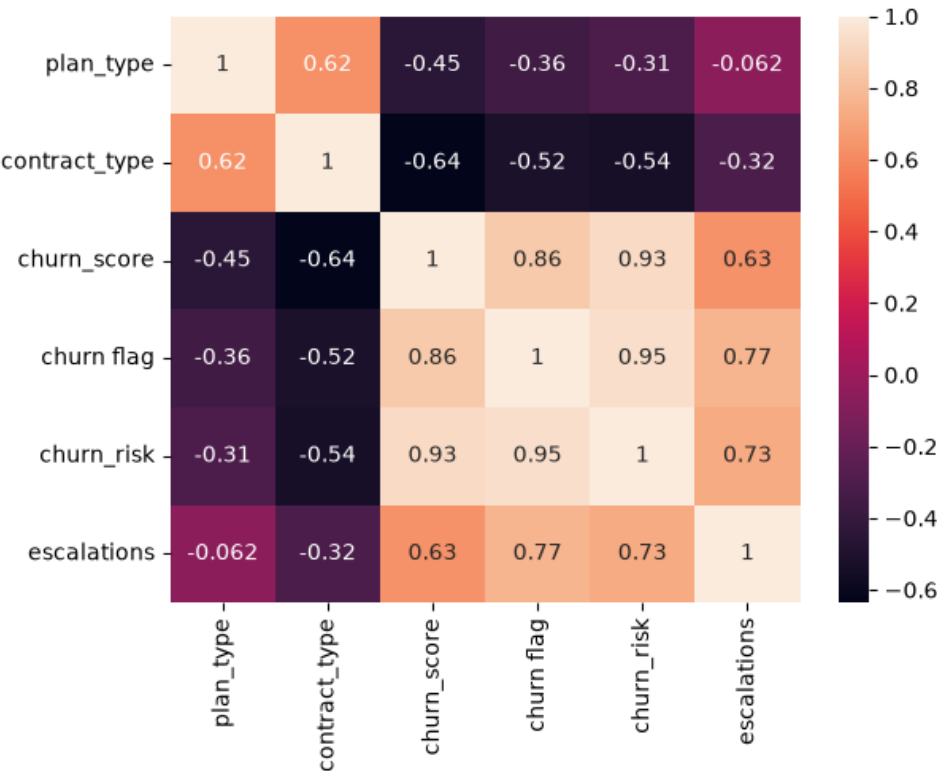
Code Cell 102

```
sns.heatmap(df_encoded.corr(),annot=True)
```

Output

<Axes: >

Visualization



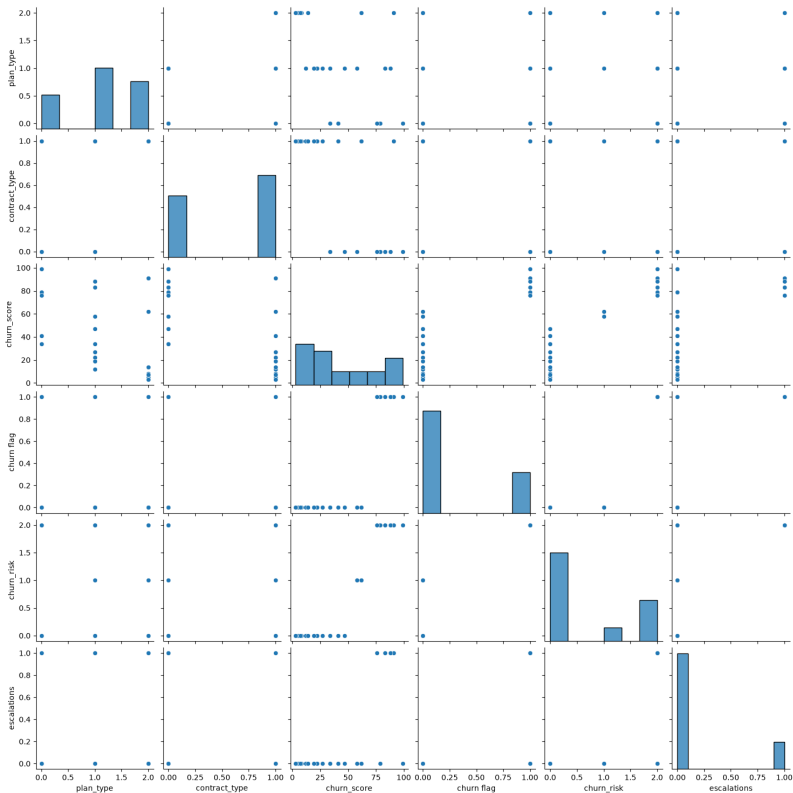
Code Cell 103

```
# pair plot
sns.pairplot(df_encoded)
```

Output

<seaborn.axisgrid.PairGrid at 0x20336ba0440>

Visualization



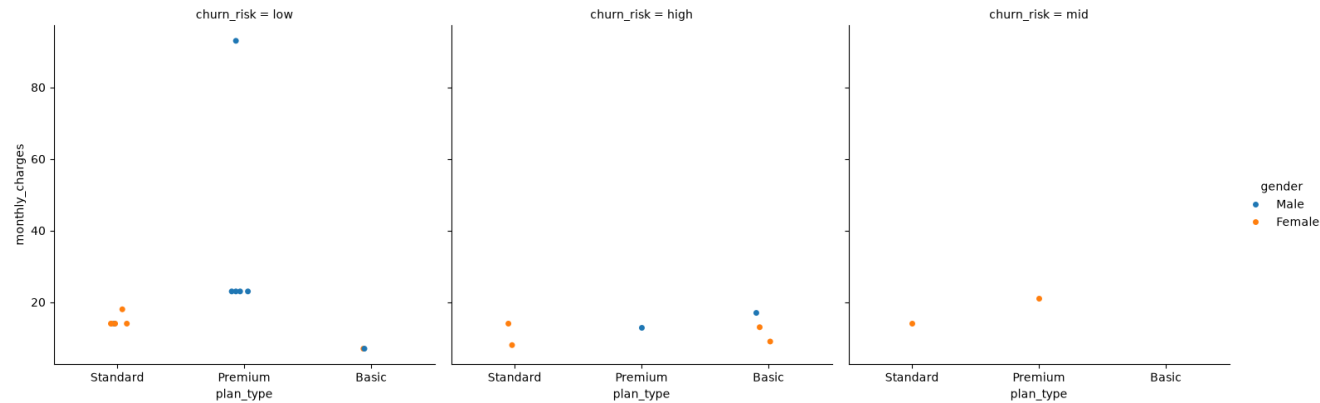
Code Cell 104

```
#catplt / Facegrid plot - mutli dim comparison
sns.catplot(data = df_vis,
            x = 'plan_type',
            y= 'monthly_charges',
            hue = 'gender',
            col = 'churn_risk')
```

Output

```
<seaborn.axisgrid.FacetGrid at 0x20338598830>
```

Visualization



Code Cell 105

```
# pivot table
pd.pivot_table(
    df_vis,
    index = 'plan_type',
    values = ['monthly_charges', 'customerid', 'churn flag'],
    aggfunc = {
        'monthly_charges': 'sum',
        'customerid': 'nunique',
        'churn flag': 'mean'
    }
)
```

Output

	churn flag	customerid	monthly_charges
plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

Code Cell 106